Detailed Work Plan (Step-by-Step)
for
Aayush Marasini
Proactive State-Aware and Thermal-Resilient Inference Scheduling for Sustained Edge Vision

Student Work Plan (PI-led)

February 6, 2026

Purpose and Expected Outcome

This work plan guides a student to implement, validate, and publish an IEEE-style systems paper on proactive, state-aware inference scheduling for sustained edge vision workloads. The project will produce:

- A reproducible experimental platform (hardware + software + logging).
- A telemetry-driven proactive scheduler with bounded-cost High-Confidence Confirmation (HCC).
- A rigorous evaluation with long-horizon runs, baselines, ablations, and statistical reporting.
- A publication-ready artifact set: code repository, datasets/scripts, plots, and an IEEE manuscript draft with figures/tables.

Scope (What the student is responsible for)

- Implement YOLOv8n inference on Raspberry Pi 5 (INT8 preferred) with controllable configuration knobs.
- Implement telemetry collection (temperature, utilization, voltage/current) with synchronized timestamps.
- Implement scheduling policy (risk-aware cost) and HCC mechanism.
- Design and execute experiments (60–120+ minute sustained runs, repeated trials).
- Produce figures, tables, and text for an IEEE submission; ensure reproducibility.

Assumptions and Constraints

- Target device: Raspberry Pi 5, active cooling, stable power supply; inline USB-C power meter available.

- Model/task: YOLOv8n on a relevant detection dataset (e.g., RDD2022 subset) and/or live camera stream.
- Telemetry access: Linux sysfs/VCGencmd for temperature and throttling flags; utilization from `psutil` or `/proc`; voltage/current from inline meter (logged via its interface or manual export).
- No kernel modifications required; use user-space scheduling.

Definition of Done (Publication Readiness Checklist)

The project is considered "done" when the following are true:

1. **Reproducible pipeline:** One command can run inference + logging + scheduling and outputs a standardized run folder.
2. **Scheduler validated:** Demonstrated proactive behavior (acts before throttle) on at least 3 long runs.
3. **Baselines complete:** Static-Max, Static-Min, Reactive Thermal Threshold, and at least one simple heuristic competitor.
4. **Ablations complete:** Telemetry signals removed one at a time; HCC removed; derivative features removed.
5. **Results plots/tables:** Time-to-throttle, FPS variance, energy per inference, mAP (or proxy), stress-event rate.
6. **IEEE paper draft:** Full manuscript with 2–4 figures, 2–3 tables, and a reproducibility paragraph.
7. **Artifact package:** Clean repo, environment file, scripts, and documented run instructions.

Project Timeline Overview

A 12-week plan is shown below. If a longer semester timeline is available, stretch evaluation and writing (Weeks 9–12) rather than implementation (Weeks 1–6).

Phase	Goal
Weeks 1–2	Platform bring-up: inference, dataset, logging, and repeatable runs.
Weeks 3–4	Configuration space + profiling: build cost/accuracy tables; establish baselines.
Weeks 5–6	Scheduler implementation: multi-modal state, derivatives, risk penalty, hysteresis.
Weeks 7–8	HCC + ablations: confirmation logic and component isolation tests.
Weeks 9–10	Long-horizon evaluation: multiple trials, stress scenarios, statistics.
Weeks 11–12	Paper-quality figures + writing: finalize IEEE manuscript and artifacts.

Step-by-Step Work Plan

Week 1: Hardware/Software Bring-Up (Make it run, make it log)

Goal: A stable, repeatable inference pipeline that runs for 30+ minutes and logs all key telemetry with synchronized timestamps.

1. **Provision device and OS**
   - Install Raspberry Pi OS (64-bit), update firmware, enable performance governor options.
   - Configure active cooling; document fan curve and ambient temperature.
   - Record exact hardware setup: Pi revision, cooling, PSU model, SD/SSD, camera model (if used).
2. **Install and validate inference stack**
   - Install Ultralytics YOLOv8 and choose backend (OpenVINO strongly preferred for INT8).
   - Confirm inference runs on sample images/video; record baseline FPS at one configuration (e.g., 640p FP32).
   - Implement a simple script: read frames → inference → write detections to JSON (timestamped).
3. **Implement telemetry logger (synchronized)**
   - Temperature $T(t)$: read from sysfs or `vcgencmd measure_temp`.
   - Throttle flags: read `vcgencmd get_throttled`.
   - Utilization $U(t)$: CPU percent (and GPU if available); collect per-core and aggregate.
   - Voltage $V(t)$ and current $I(t)$: log from inline meter (export file) or manual capture; ensure timestamps.
   - Sampling: start at 1–5 Hz; make adjustable.
   - Output: single CSV with a run ID and UTC timestamps.
4. **Run-folder standardization**
   - Every run creates a folder: `runs/YYYYMMDD_HHMMSS_configX/`.
   - Save: config, commit hash, environment info, telemetry CSV, inference logs, optional video snippet.
   - Add a script `summarize_run.py` that computes basic metrics (avg FPS, max temp).
5. **Deliverables (end of Week 1)**
   - Repo with `run_static.py` (fixed config) and `log_telemetry.py`.
   - One successful 30-minute run folder with complete logs.
   - Short platform report (1 page) describing setup and achieved FPS/temp behavior.

Week 2: Dataset, Ground Truth, and Metrics Definition (Make evaluation defensible)

Goal: Define what "accuracy" means in sustained runs and how it will be computed or approximated.

1. **Select evaluation input streams**
   - Option A (recommended): curated video clips representing deployment conditions (day/night, motion, clutter).
   - Option B: live camera with recorded sessions; ensure reproducibility by saving the input stream.
   - Ensure at least 3 distinct workload scenarios: *easy*, *moderate*, *hard*.
2. **Define accuracy measurement strategy**
   - If full mAP is feasible: run periodic evaluation windows (e.g., every 5 minutes evaluate on a labeled batch).
   - If full mAP is not feasible in real time: define a proxy metric (e.g., agreement with high-fidelity reference).
   - Define a *reference configuration* (e.g., 640p INT8 or FP32) as "oracle" for agreement checks.
3. **Define stability and stress metrics**
   - FPS mean and variance; percentile FPS (P5, P50, P95).
   - Time-to-throttle: first time throttle flag indicates downclocking.
   - Thermal headroom: time spent above 70°C and above 75°C.
   - Electrical stress: time spent with $V(t)$ below threshold; count of voltage-sag events.
   - Energy metrics: joules per inference; joules per minute; energy per confirmed event.
4. **Deliverables (end of Week 2)**
   - A `metrics.md` defining all metrics and how computed.
   - A script `compute_metrics.py` that outputs a JSON summary from a run folder.
   - A fixed, versioned input set (videos or recorded stream) for reproducible experiments.

Week 3: Configuration Space and Profiling (Build the cost and accuracy tables)

Goal: Measure $E(c)$, throughput, and accuracy for each configuration $c = \{r, p, f\}$.

1. **Enumerate configurations**
   - Resolutions: e.g., $\{160, 224, 320, 416, 640\}$.
   - Precision: FP32 and INT8 (and FP16 if available).
   - FPS caps: e.g., $\{5, 10, 15, 30\}$ or as feasible.
2. **Micro-benchmark each configuration**
   - For each c, run a 2–5 minute fixed-input loop and measure: FPS, average power, max temperature slope, energy per inference.

- Store results in `profiles/config_table.csv`.

3. **Accuracy profiling**
   - Evaluate each configuration on a labeled validation set or via agreement with reference.
   - Produce an `accuracy_table.csv` giving $A(c)$.
4. **Pareto frontier and candidate set**
   - Plot accuracy vs energy; identify dominated configurations.
   - Select a candidate set $\mathcal{C}' \subset \mathcal{C}$ to reduce runtime overhead.
5. **Deliverables (end of Week 3)**
   - Configuration profiling tables for energy/FPS/temperature slope and accuracy.
   - One figure: accuracy-energy Pareto plot (paper-ready style).

Week 4: Baseline Implementations (So comparisons are fair)

Goal: Implement baselines in the same pipeline and verify correctness.

1. **Static baselines**
   - Static-Max: highest fidelity config (e.g., 640p, best precision, uncapped FPS or highest cap).
   - Static-Min: low-power config (e.g., 320p INT8 with FPS cap).
2. **Reactive thermal threshold baseline**
   - Define thresholds (e.g., switch down at 75°C, switch up at 70°C).
   - Add hysteresis to avoid oscillation; log every switch event.
3. **Simple heuristic competitor (recommended)**
   - Utilization-only reactive: switch down when $U(t) > U_{hi}$, up when $U(t) < U_{lo}$.
   - This helps demonstrate the value of multi-modal derivatives beyond naive heuristics.
4. **Validation**
   - Run 20-minute tests for each baseline; confirm logs and switching events align with design.
5. **Deliverables (end of Week 4)**
   - Baseline scripts integrated: `run_baseline_static.py`, `run_baseline_thermal.py`, etc.
   - Baseline sanity results: short report + plots.

Week 5: Scheduler Core (State, derivatives, risk)

Goal: Implement the proactive state-aware scheduler with derivative-based lead indicators.

1. **State estimation**
   - Implement real-time state vector $s(t) = [T, \dot{T}, U, \dot{U}, V, \dot{V}]$.
   - Use smoothing (e.g., EWMA) to reduce noise; document chosen parameters.
   - Compute derivatives via finite differences with consistent sampling period.
2. **Risk model $\Phi(c, t)$ (start simple, then refine)**
   - Begin with a heuristic risk score:

     $$R(t) = w_T \cdot \sigma(T) + w_{\dot{T}} \cdot \sigma(\dot{T}) + w_U \cdot \sigma(U) + w_{\dot{U}} \cdot \sigma(\dot{U}) + w_V \cdot \sigma(-\dot{V}),$$

     where $\sigma(\cdot)$ maps values to $[0, 1]$.
   - Define $\Phi(c, t) = \lambda(c) \cdot R(t)$ where $\lambda(c)$ reflects how "expensive" the config is.
   - Add guardrails: do not switch up if $T > T_{up_max}$, etc.
3. **Decision logic and hysteresis**
   - Add minimum dwell time between switches (e.g., 3–10 seconds).
   - Add two-threshold logic (up/down thresholds) to avoid flapping.
   - Log every state estimate and action decision with reasons.
4. **Unit tests / simulation harness**
   - Build an offline "replay" tool that reads telemetry logs and simulates decisions.
   - This accelerates tuning without re-running long experiments.
5. **Deliverables (end of Week 5)**
   - `scheduler.py` implementing state estimation and decision policy.
   - Offline replay/tuning tool + unit tests.
   - One plot showing proactive downshift occurs before throttle in a short stress run.

Week 6: Scheduler Tuning + Safety (Make it robust)

Goal: Tune thresholds/weights and validate robustness across scenarios.

1. **Parameter tuning protocol**
   - Tune on Scenario A only; reserve Scenarios B/C for validation.
   - Use objective: maximize time-to-throttle and minimize FPS variance while preserving accuracy proxy.
2. **Stress scenario design**
   - Induce thermal stress: reduce fan speed or increase ambient temperature.
   - Induce electrical stress: use long USB-C cable or lower PSU margin (safely).

- Induce compute stress: add background load to increase utilization.

3. **Failure mode checks**
   - Ensure scheduler never violates critical limits (e.g., no prolonged operation above 75°C).
   - Ensure scheduler does not oscillate excessively (cap switch rate).
   - Ensure graceful behavior when telemetry temporarily missing.
4. **Deliverables (end of Week 6)**
   - Tuned scheduler parameters with a documented rationale.
   - Validation runs in Scenarios B/C showing consistent stability improvements.

Week 7: High-Confidence Confirmation (HCC) Implementation

Goal: Add event-driven confirmation to protect accuracy in low-power operation.

1. **Define trigger for HCC**
   - Trigger candidates: low-confidence detections, rare-class detection, motion/scene change, or anomaly score.
   - Keep trigger lightweight and deterministic (avoid heavy compute).
2. **HCC execution**
   - When triggered, run N_{HCC} frames at higher-fidelity config; then return to base.
   - Log trigger reason, start/end times, energy overhead, and confirmation outcome.
3. **Bounded-cost verification**
   - Compute ΔE_{HCC} and show it is small relative to sustained high-power baseline.
   - Evaluate trade-off: HCC improves accuracy proxy with minimal stability loss.
4. **Deliverables (end of Week 7)**
   - Working HCC module integrated with scheduler.
   - One figure: example trace showing HCC bursts and negligible thermal impact.

Week 8: Ablations (Prove which parts matter)

Goal: Produce rigorous ablation results that justify each component.

1. **Telemetry ablations**
   - Remove each signal family and re-run: (i) thermal-only, (ii) thermal+utilization, (iii) thermal+voltage, (iv) full.
   - Remove derivatives (use only absolute values) and compare.
2. **Policy ablations**
   - Remove risk penalty term Φ (accuracy/energy only).

- Remove hysteresis/dwell-time (show oscillation risk).
- Remove HCC (show accuracy vs energy effect).

3. **Deliverables (end of Week 8)**
   - Ablation table with 6–10 rows and key metrics.
   - Short interpretation notes (what each component contributes).

Weeks 9–10: Long-Horizon Evaluation (Make it publishable)

Goal: Execute the main evaluation with multiple long runs and statistical reporting.

1. **Experiment design**
   - Duration: 60–120 minutes per run (or until throttle/stability failure).
   - Repetitions: at least $n = 5$ runs per method per scenario (minimum).
   - Randomness control: fixed seeds, fixed input order, stable environment.
2. **Run matrix**
   - Methods: Static-Max, Static-Min, Thermal-Threshold, Utilization-Heuristic, Proposed Scheduler, Proposed+HCC.
   - Scenarios: Easy/Moderate/Hard; optional stress variants (thermal/electrical/compute).
3. **Statistical reporting**
   - Report mean $\pm$ standard deviation; include confidence intervals where appropriate.
   - Use nonparametric tests if distributions are non-normal; document method.
   - Include effect sizes (e.g., percent improvement in time-to-throttle).
4. **Deliverables (end of Week 10)**
   - Full results dataset (run folders) and an index CSV.
   - Paper-ready plots (vector PDFs) and tables (LaTeX).

Weeks 11–12: IEEE Paper Packaging (Writing + figures + reproducibility)

Goal: Produce a near-submission-ready IEEE paper and an artifact package.

1. **Figures to produce (minimum set)**
   - Fig. 1: System overview (telemetry $\rightarrow$ state $\rightarrow$ scheduler $\rightarrow$ config).
   - Fig. 2: Example time-series trace (T, U, V, FPS, config switches) for one long run.
   - Fig. 3: Main results summary (bar/line plots for time-to-throttle, FPS variance, energy, accuracy).
   - Optional Fig. 4: Ablation results or HCC trace.
2. **Tables to produce**
   - Table 1: Configuration set and profiling summary.

- Table 2: Main long-horizon performance metrics across methods.
- Optional Table 3: Ablations.

3. **Writing tasks**
   - Write a crisp contributions paragraph in the introduction (3 bullet contributions).
   - Add a reproducibility paragraph: device, OS, versions, how to run experiments.
   - Ensure related work cites 8–15 relevant papers (edge inference, thermal mgmt, selective inference).
4. **Artifact packaging**
   - Clean repository: `README`, environment setup, run scripts, config files.
   - Add `scripts/make_plots.py` that regenerates every figure from logs.
   - Add `LICENSE` and `CITATION.cff` (optional but recommended).
5. **Deliverables (end of Week 12)**
   - IEEE paper draft with final figures/tables inserted.
   - Artifact package for internal review; optionally prepare for open-source release.

Milestones and Deliverables Table

Week	Milestone	Deliverables
1	Pipeline runs + telemetry logging	Repo scripts; 30-min run folder; platform report
2	Metrics and evaluation protocol locked	`metrics.md`; `compute_metrics.py`; versioned inputs
3	Config profiling complete	Energy/FPS/accuracy tables; Pareto plot
4	Baselines implemented	Baseline scripts; sanity plots
5	Scheduler core implemented	`scheduler.py`; replay tool; proactive trace
6	Scheduler tuned and robust	Parameter doc; validation runs
7	HCC integrated	HCC module; trace figure
8	Ablations complete	Ablation table + notes
9–10	Long-horizon evaluation complete	Run index; paper-ready plots/tables
11–12	IEEE paper + artifacts near-final	Full draft; figures; reproducibility + scripts

Risk Register and Mitigation

- **Telemetry noise or missing signals**: add smoothing; design fallbacks; log sensor health; reduce reliance on any single signal.
- **Voltage logging difficulty**: if inline meter export is limited, use a different meter or log power via a smart plug as a fallback (note lower fidelity).
- **mAP computation overhead**: use periodic evaluation windows or oracle-agreement proxy; document limitations transparently.
- **Thermal environment variability**: measure ambient temperature; run tests at consistent times; report variance.
- **Oscillation / instability in scheduling**: enforce dwell time and hysteresis; cap switch frequency; validate under stress scenarios.

Recommended Weekly Meeting Structure (PI-led)

Each week, the student should deliver (i) a one-page progress memo, (ii) a short demo (script or plot), and (iii) a next-week risk list. The PI should review results plots early (Weeks 3–4) to ensure the evaluation story will be publishable before heavy tuning begins.